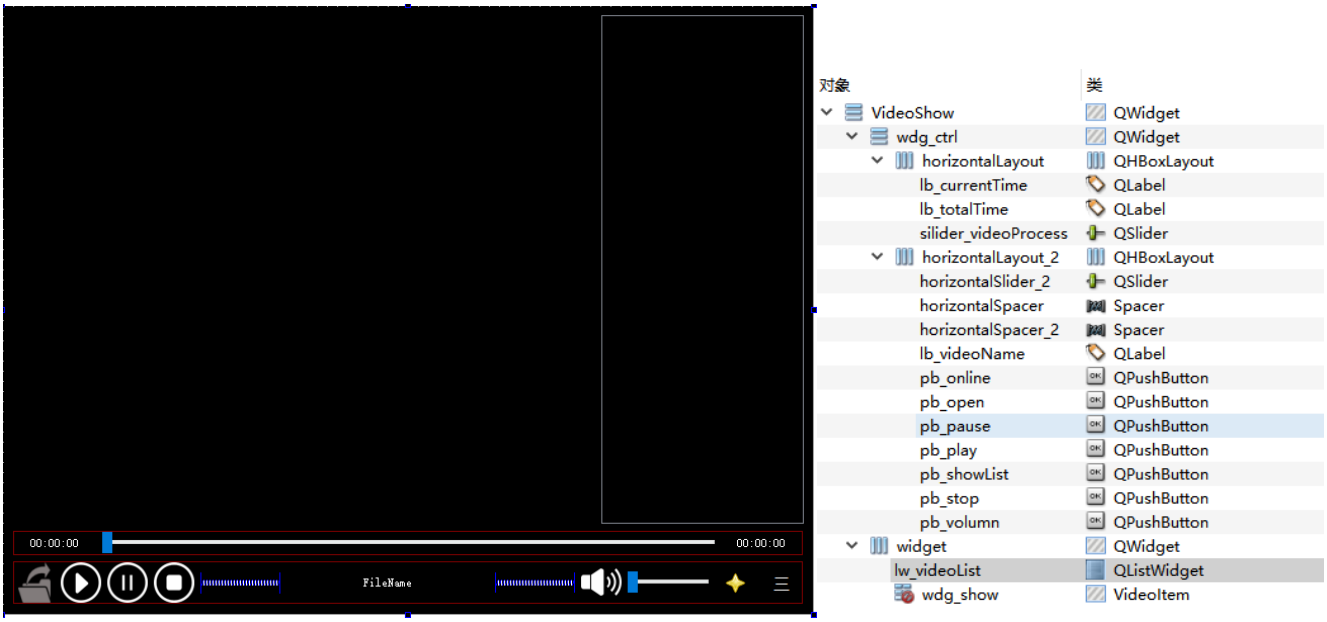
六.音视频播放控制

接下来做播放的控制. 添加的功能包括, 打开文件, 播放, 暂停, 进度显示和调整, 播放时间显示.

1.界面调整

首先在界面中添加控件.



添加一系列控件, 分别是打开文件, 播放, 暂停, 停止, 播放进度条, 音量进度条等.

然后我们去添加信号对应的槽函数.

```
    void on_pb_play_clicked();
    void on_pb_open_clicked();
    void on_pb_pause_clicked();
    void on_pb_stop_clicked();
```

先完成 VideoPlayer 的修改.

在 VideoPlayer 类里面添加成员

```
//////////////////////////
///播放控制
void play();
void pause();
void stop( bool isWait);
void setFileName(const QString &fileName);
PlayerState m_playerState;
```

为了完成播放的控制, 需要在 VideoPlayer 中添加成员 PlayerState m_playerState;播放状态是一个枚举, 代表着播放状态.

```
enum PlayerState
{
    Playing = 0,
    Pause,
    Stop
};
```

除此之外，需要对 VideoState 结构体进行修改来控制播放

```
class VideoPlayer ;
typedef struct VideoState {
    AVFormatContext *pFormatCtx;//相当于视频”文件指针”
    ///////////////音频/////////////////////////////////
    AVStream    *audio_st; //音频流
    PacketQueue *audioq;//音频缓冲队列
    AVCodecContext *pAudioCodecCtx ;//音频解码器信息指针
    int audioStream;//视频音频流索引
    double audio_clock; ///<pts of last decoded frame 音频时钟
    SDL_AudioDeviceID audioID; //音频 ID
    AVFrame out_frame; //设置参数，供音频解码后的 swr_alloc_set_opts 使用。
    ////////////////////////////////////////////////
    ///////////////视频/////////////////////////////////
    AVStream    *video_st; //视频流
    PacketQueue *videoq;//视频队列
    AVCodecContext *pCodecCtx ;//音频解码器信息指针
    int videoStream;//视频音频流索引
    double video_clock; ///<pts of last decoded frame 视频时钟
    SDL_Thread *video_tid;  //视频线程 id
    ////////////////////////////////////////////////
    /// 播放控制的变量
    bool isPause;//暂停标志
    bool quit; //停止
    bool readFinished; //读线程文件读取完毕
    bool readThreadFinished; //读取线程是否结束
    bool videoThreadFinished; // 视频线程是否结束
    ////////////////////////////////////////////////
    //// 跳转相关的变量
    int             seek_req; //跳转标志 -- 读线程
    int64_t         seek_pos; //跳转的位置 -- 微秒
    int             seek_flag_audio;//跳转标志 -- 用于音频线程中
    int             seek_flag_video;//跳转标志 -- 用于视频线程中
    double          seek_time; //跳转的时间(秒)  值和 seek_pos 是一样的
    ////////////////////////////////////////////////
    int64_t start_time; //单位 微秒
    VideoState()
    {
        audio_clock = video_clock  = start_time = 0;
    }
    VideoPlayer* m_player;//用于调用函数
} VideoState;
```

2.打开文件

```
m_videoState.readThreadFinished = true;
```

上面的函数 void setFileName(const QString &fileName);并不只是设置路径，还要直接开始播放.

```
void VideoPlayer::setFileName(const QString &fileName)
{
    if( m_playerState != PlayerState::Stop  )   return;
    m_fileName = fileName;
    m_playerState = PlayerState::Playing;
    this->start();
}
```

在 ui 类中添加成员

```
public:
    int isStop;//代表当前是否是停止状态
```

ui 类中添加状态切换槽函数

```
//播放状态切换槽函数
void VideoShow::slot_PlayerStateChanged(int state)
{
     switch( state )
     {
        case PlayerState::Stop:
            qDebug()<< "VideoPlayer::Stop";
            m_Timer->stop();
            ui->slider_videoProcess->setValue(0);
            ui->lb_totalTime->setText("00:00:00");
            ui->lb_currentTime->setText("00:00:00");
            ui->lb_videoName->setText("");
            ui->pb_pause->hide();
            ui->pb_play->show();
            {
                 QImage img;
                 img.fill( Qt::black);
                 ui->wdg_video->slot_setImage( img );
            }
            this->update();
            isStop = true;
        break;
        case PlayerState::Playing:
             qDebug()<< "VideoPlayer::Playing";
             ui->pb_play->hide();
             ui->pb_pause->show();
             m_Timer->start();
             this->update();
             isStop = false;
        break;
     }
}
```

构造里面，播放状态应该是 stop.

slot_PlayerStateChanged(PlayerState::Stop);

添加打开文件函数的实现

```
#include<QFileDialog>
#include<QMessageBox>
//打开文件
void VideoShow::on_pb_open_clicked()
{
    //打开文件 弹出对话框 参数:父窗口, 标题, 默认路径, 筛选器
    QString path = QFileDialog::getOpenFileName(this,"选择要播放的文件" , "F:/",
            "视频文件 (*.flv *.rmvb *.avi *.MP4 *.mkv);; 所有文件(*.*);;");
    if(!path.isEmpty())
    {
        qDebug()<< path ;
        QFileInfo info(path);
        if( info.exists() )
        {
            m_player->stop( true ); //如果播放 你要先关闭
            m_player->setFileName(path);
            ui->lb_videoName->setText( info.baseName() );
            slot_PlayerStateChanged(PlayerState::Playing);
        }
        else
        {
            QMessageBox::information( this, "提示" , "打开文件失败");
        }
    }
}
```

3.播放暂停

接下来, 来完成暂停的功能.
暂停需要一个状态,在结构体 VideoState 中 isPause 来表示, 用这个变量,来控制三个线程(读文件线程 run() , 音频解码线程, 视频解码线程)的暂停, 方法很简单就是在循环的位置, 添加判断

```
if( m_videoState.isPause )
{
    SDL_Delay(10);
    continue;
}
```

比如:
run 线程中

```
while(1)
{
    if( m_videoState.quit ) break;
    if( m_videoState.isPause == true )
    {
        SDL_Delay(10);
        continue;
    }
```

视频线程中

```
if( is->quit ) break;
if( is->isPause )
{
    SDL_Delay(10);
    continue;
}
```

音频回调函数解码函数中:

```
for(;;)
{
    if( is->quit ) return -1;

    if( is->isPause == true )
    {
        SDL_Delay(10);
        continue;
    }
```

然后添加 VideoPlayer 的函数 pause()的实现.

```
//暂停
void VideoPlayer::pause()
{
    m_videoState.isPause = true;
    if( m_playerState != Playing ) return;
    m_playerState = Pause;
}
```

然后添加 VideoPlayer 的函数 play()的实现.

```
void VideoPlayer::play()
{
    m_videoState.isPause = false;
    if( m_playerState != Pause) return;
    m_playerState = Playing;
}
```

暂停播放时, 需要在回调函数中加入清空缓冲区 , 为了避免一直播放上一帧声音

```
memset( stream , 0 , len);
if(is->isPause ) return;
```

如图

```
//回调函数中将从队列中取数据, 解码后填充到播放缓冲区.
void audio_callback(void *userdata, Uint8 *stream, int len)
{
//    AVCodecContext *pcodec_ctx    = (AVCodecContext *) userdata;
    VideoState * is = (VideoState *) userdata;

    int len1, audio_data_size;

    memset( stream , 0 , len);
    if( is->isPause ) return;
//    static uint8_t audio_buf[(AVCODEC_MAX_AUDIO_FRAME_SIZE * 3) / 2];
```

ui 控制类中的播放,暂停

```
//暂停
void VideoShow::on_pb_pause_clicked()
{
    if( isStop ) return;//停止状态自然不能播放
```

```
    m_Player->pause();

    if( m_Player->getPlayerState() == PlayerState::Pause)
    {
        ui->pb_pause->hide();
        ui->pb_play->show();

        this->update();
    }
}
//播放
void VideoShow::on_pb_play_clicked()
{
    if( isStop ) return;
    m_Player->play();
    if(m_Player->getPlayerState() == PlayerState::Playing)
    {
        ui->pb_play->hide();
        ui->pb_pause->show();
        this->update();
    }
}
```

这里的播放时暂停到播放，并不是停止到播放，该项目中不会写播放列表，想要完善播放列表,可以参考音乐播放器的写法.

然后对 run 函数进行修改，我们需要把 ffmpeg 和 SDL 的初始化移动到控件的构造的位置.

4.播放停止

读取进程 run()函数修改

读取数据包时,发现返回<0，并不需要马上退出，可以做一下延迟

```
if (av_read_frame(pFormatCtx, packet) < 0)
{
    DelayCount++;
    if( DelayCount>= 300)
    {
        m_videoState.readFinished = true;
        DelayCount = 0 ;
    }

    if( m_videoState.quit) break; //解码线程执行完 退出

    SDL_Delay(10);
    continue;
}
```

DelayCount = 0;

当退出循环时,等待音频和视频解码结束

```
while( !m_videoState.quit)
{
    SDL_Delay(100);
}
//回收空间
if( m_videoState.videoStream != -1)
    packet_queue_flush( m_videoState.videoq);//队列回收
if( m_videoState.audioStream != -1)
    packet_queue_flush( m_videoState.audioq); //队列回收
while( m_videoState.videoStream != -1 && !m_videoState.videoThreadFinished )
{
    SDL_Delay(10);
}
if( audioStream != -1)
{
    //回收空间
    avcodec_close(pAudioCodecCtx);
}
    //9.回收数据
if( videoStream != -1 )
{
    avcodec_close(pCodecCtx);
}
avformat_close_input(&pFormatCtx);
//回收资源之后,在最后添加读取文件线程退出标志.
m_videoState.readThreadFinished = true;
//视频自动结束 置标志位
m_playerState = PlayerState::Stop;
```

其中，队列回收函数

```
void packet_queue_flush(PacketQueue *q)
{
    AVPacketList *pkt, *pkt1;

    SDL_LockMutex(q->mutex);
    for(pkt = q->first_pkt; pkt != NULL; pkt = pkt1)
    {
        pkt1 = pkt->next;
        av_free_packet(&pkt->pkt);
        av_freep(&pkt);
    }
    q->last_pkt = NULL;
    q->first_pkt = NULL;
    q->nb_packets = 0;
    q->size = 0;
    SDL_UnlockMutex(q->mutex);
}
```

视频解码线程修改:
从队列中读取数据需要改成

```
if (packet_queue_get(is->videoq, packet, 0) <= 0)
{
    if( is->readFinished && is->audioq->nb_packets == 0)//播放到结束
    {//读线程完毕
        break;
    }else
    {
        SDL_Delay(1);    //只是队列里面暂时没有数据而已
        continue;
    }
}
```

```
  if( is->quit ) break;
  if( is->isPause )
  {
      SDL_Delay(10);
      continue;
  }
  if (packet_queue_get(is->videoq, packet, 0) <= 0)
  {
      if( is->readFinished && is->audioq->nb_packets == 0)
      {//读线程完毕
          break;
      }else
      {
          SDL_Delay(1);    //只是队列里面暂时没有数据而已
          continue;
      }
  }
```

在 while(1)循环的流程里, 添加 quit 标志位退出

```
while(1)
{
    if( is->quit) break;
    if( is ->audioq->size == 0   ) break;
    audio_pts = is->audio_clock;
    video_pts = is->video_clock;
    if (video_pts <= audio_pts) break;
    SDL_Delay(5);
}
```

在退出函数之前, 设置标志位

```
if( !is->quit)
{
    is->quit = true;
}
av_free(pFrame);
av_free(pFrameRGB);
av_free(out_buffer_rgb);
is->videoThreadFinished = true;

// 清屏
QImage img; //把图像复制一份 传递给界面显示
```

```
img.fill(Qt::black);
is->m_player->SendGetOneImage(img); //调用激发信号的函数
```

音频解码函数的停止: 在从队列中读取时, 添加退出标志

```
if(packet_queue_get(audioq, &pkt, 0) <= 0) //一定注意
{
    if( is->readFinished && is->audioq->nb_packets == 0 )
        is->quit = true;
    return -1;
}
//播放停止
void VideoPlayer::stop(bool isWait)//阻塞标志
{
    m_videoState .quit = 1;
    if( isWait ) //阻塞标志
    {
       while(!m_videoState.readThreadFinished )//等待读取线程退出
       {
           SDL_Delay(10);
       }
    }
    //关闭 SDL 音频设备
    if (m_videoState.audioID != 0)
    {
        SDL_LockAudio();
        SDL_PauseAudioDevice(m_videoState.audioID,1);//停止播放,即停止音频回调函数
        SDL_CloseAudioDevice( m_videoState.audioID );
        SDL_UnlockAudio();
        m_videoState.audioID = 0;
    }
    m_playerState = PlayerState::Stop;
    Q_EMIT SIG_PlayerStateChanged(PlayerState::Stop);
}
```

这里 isWait 这个参数主要是用来判断是否要等待进程完成读取.

在 VideoPlayer 类中添加信号定义

```
void SIG_PlayerStateChanged(int state);
```

在 ui 类中添加处理该信号的槽函数连接

```
connect( m_Player, SIGNAL( SIG_PlayerStateChanged(int))  ,
            this ,SLOT(slot_PlayerStateChanged(int)) );
```

注意不要使用 PlayerState 类型名,因为 connect 连接中的参数最好不要出现自定义类型,不然可能出问题.

ui 界面类调整

```
//停止
void VideoShow::on_pb_stop_clicked()
{
```

```
    m_Player->stop(true);
}
```

5.视频进度条

首先是读取视频的时间长度.获取当前时间和获取总时间

```
//获取当前时间
double VideoPlayer::getCurrentTime()
{
    return m_videoState.audio_clock;
}
//获取总时间
int64_t VideoPlayer::getTotalTime()
{
    if( m_videoState.pFormatCtx )
    return m_videoState.pFormatCtx->duration;
    return -1;
}
```

他们的单位都是微秒

然后在 VideoPlayer 中定义信号 void SIG_TotalTime(qint64 uSec); 读取文件进程函数 run()函数中, 一旦开始解析视频文件, 发送此信号

```
Q_EMIT SIG_TotalTime( getTotalTime() );
```

然后在 ui 显示界面类中, 添加

```
connect( m_Player, SIGNAL( SIG_TotalTime(qint64))  ,
            this ,SLOT( slot_getTotalTime(qint64)) );
//获取视频全部时间
void VideoShow::slot_getTotalTime(qint64 uSec)
{
    qint64 Sec = uSec/1000000;

    ui->slider_videoProcess->setRange(0,Sec);//精确到秒

    QString hStr = QString("00%1").arg(Sec/3600);
    QString mStr = QString("00%1").arg(Sec/60);
    QString sStr = QString("00%1").arg(Sec%60);

    QString str =
QString("%1:%2:%3").arg(hStr.right(2)).arg(mStr.right(2)).arg(sStr.right(2));
    ui->lb_totalTime->setText(str);
}
```

申请定时器成员

```
QTimer *m_Timer;
m_Timer = new QTimer; //定时器-获取当前视频时间
connect(m_Timer,SIGNAL(timeout()),this,SLOT(slot_TimerTimeOut()));
```

```
m_Timer->setInterval(500);
```

添加定时器超时的响应函数 void slot_TimerTimeOut();

```
//获取当前视频时间定时器
void VideoShow::slot_TimerTimeOut()
{
    if (QObject::sender() == m_Timer)
    {

        qint64 Sec = m_Player->getCurrentTime()/1000000;

        ui->slider_videoProcess->setValue(Sec);

        QString hStr = QString("00%1").arg(Sec/3600);
        QString mStr = QString("00%1").arg(Sec/60%60);
        QString sStr = QString("00%1").arg(Sec%60);

        QString str =
QString("%1:%2:%3").arg(hStr.right(2)).arg(mStr.right(2)).arg(sStr.right(2));
        ui->lb_currentTime->setText(str);
        if(ui->slider_videoProcess->value() == ui->slider_videoProcess->maximum()
                && m_Player->m_playerState == PlayerState::Stop)
        {
            slot_PlayerStateChanged( PlayerState::Stop );
        }else if(ui->slider_videoProcess->value() + 1  ==
ui->slider_videoProcess->maximum()
                && m_Player->m_playerState == PlayerState::Stop)
        {
            slot_PlayerStateChanged( PlayerState::Stop );
        }
    }
}
```

6.跳转功能

和暂停一样，我们使用一个变量记录下是否需要跳转，在读取视频文件的线程中判断此变量，当需要跳转的时候就执行跳转操作。

跳转操作可以直接使用 FFMPEG 的跳转函数 av_seek_frame 来实现，函数原型如下：

```
int av_seek_frame(AVFormatContext *s, int stream_index, int64_t timestamp,int flags);
```

这里需要注意三个问题：

第一个问题

我们有 2 个队列，音频队列和视频队列，这 2 个队列里面的数据是可以播放几秒钟的，因此每次执行跳转的时候需要同时将队列清空，所以先添加一个清空队列的函数：

```
void  packet_queue_flush(PacketQueue  *q);
```
然后每次调用 seek 的时候便执行一次清空操作：
```
if  (av_seek_frame(is->ic,  stream_index,  0,is->seek_flags)  <  0)
{        fprintf(stderr,  "%s:  error  while  seeking",is->ic->filename);
}
else  {
        if  (is->audioStream  >=  0)  {
                packet_queue_flush(&is->audioq);
        }
        if  (is->videoStream  >=  0)  {
                packet_queue_flush(&is->videoq);
        }
}
```

第二个问题

按照上面的方式处理，每次跳转的时候都会出现花屏的现象。
是因为解码器中保留了上一帧视频的信息，而现在视频发生了跳转，从而使得解码器中保留的信息会对解码当前帧产生影响。
因此，清空队列的时候我们也要同时清空解码器的数据(包括音频解码器和视频解码器)，
可以往队列中放入一个特殊的 packet，当解码线程取到这个 packet 的时候，就执行清除解码器的数据：
```
#define FLUSH_DATA "FLUSH"
在读取文件线程 run()函数里面添加
if( m_videoState.seek_req )
// 跳转标志位 seek_req --> 1 清除队列里的缓存 3s --> 3min 3s 里面的数据 存在 队列和解码器
// 3s 在解码器里面的数据和 3min 的会合在一起 引起花屏 --> 解决方案 清理解码器缓存
AV_flush_...
//什么时候清理 -->要告诉它 , 所以要来标志包 FLUSH_DATA "FLUSH"
//关键帧--比如 10 秒 --> 15 秒 跳转关键帧 只能是 10 或 15 , 如果你要跳到 13 , 做法是跳到 10 然后 10-13 的包全扔掉
{
    int stream_index = -1;
    int64_t seek_target = m_videoState.seek_pos;//微秒

    if (m_videoState.videoStream >= 0)
        stream_index = m_videoState.videoStream;
    else if (m_videoState.audioStream >= 0)
        stream_index = m_videoState.audioStream;

    AVRational aVRational = {1, AV_TIME_BASE};
    if (stream_index >= 0) {
        seek_target = av_rescale_q(seek_target, aVRational,
                pFormatCtx->streams[stream_index]->time_base); //跳转到的位置
    }
```

```
    if (av_seek_frame(m_videoState.pFormatCtx, stream_index, seek_target,
AVSEEK_FLAG_BACKWARD) < 0) {
        fprintf(stderr, "%s: error while seeking\n",m_videoState.pFormatCtx->filename);
    } else {
        if (m_videoState.audioStream >= 0) {
            AVPacket *packet = (AVPacket *) malloc(sizeof(AVPacket)); //分配一个 packet
            av_new_packet(packet, 10);
            strcpy((char*)packet->data,FLUSH_DATA);
            packet_queue_flush(m_videoState.audioq); //清除队列
            packet_queue_put(m_videoState.audioq, packet); //往队列中存入用来清除的包
        }
        if (m_videoState.videoStream >= 0) {
            AVPacket *packet = (AVPacket *) malloc(sizeof(AVPacket)); //分配一个 packet
            av_new_packet(packet, 10);
            strcpy((char*)packet->data,FLUSH_DATA);
            packet_queue_flush(m_videoState.videoq); //清除队列
            packet_queue_put(m_videoState.videoq, packet); //往队列中存入用来清除的包
            m_videoState.video_clock = 0; //考虑到向左快退  避免卡死
            //视频解码过快会等音频 循环 SDL_Delay 在循环过程中 音频时钟会改变 , 快退 音
频时钟变小
        }
    }
    m_videoState.seek_req = 0;
    m_videoState.seek_time = m_videoState.seek_pos ; //精确到微妙 seek_time 是用来做视
频音频的时钟调整 --关键帧
    m_videoState.seek_flag_audio = 1; //在视频音频循环中 , 判断, AVPacket 是 FLUSH_DATA
清空解码器缓存
    m_videoState.seek_flag_video = 1;
}
```

然后在解码线程中如下操作(以解码图像线程为例，音频线程也是类似)：

```
//收到这个数据 说明刚刚执行过跳转 现在需要把解码器的数据 清除一下
读取一帧数据后 比较, 是清空数据帧, 那么清除解码器中的数据
```

视频:

```
        if(strcmp((char*)packet->data,FLUSH_DATA) == 0)
        {
            avcodec_flush_buffers(is->video_st->codec);
            av_free_packet(packet);
            is->video_clock = 0;  //很关键 , 不清空 向左跳转, 视频帧会等待音频帧
            continue;
        }
```

添加位置如图

```
    if (packet_queue_get(is->videoq, packet, 0) <= 0)
    {
        if( is->readFinished && is->audioq->nb_packets == 0)
        {//读线程完毕
            break;
        }else
        {
            SDL_Delay(1);    //只是队列里面暂时没有数据而已
            continue;
        }
    }
    if( strcmp((char*)packet->data , FLUSH_DATA ) == 0 )
    {
        avcodec_flush_buffers(is->video_st->codec);
        av_free_packet(packet);
        is->video_clock = 0;
        continue;
    }
```

音频:

```
        if(strcmp((char*)pkt.data,FLUSH_DATA) == 0)
        {
            avcodec_flush_buffers(is->audio_st->codec);
            av_free_packet(&pkt);
            continue;
        }
```

添加位置如图

```
    if( audioq && packet_queue_get(audioq, &pkt, 0) <= 0) //一定注意
    {
        if( is->readFinished && is->audioq->nb_packets == 0 )
                is->quit = true;

        return -1;
    }
    if( strcmp((char*)pkt.data , FLUSH_DATA ) == 0 )
    {
        avcodec_flush_buffers(is->audio_st->codec);
        av_free_packet(&pkt);
        continue;
    }
```

现在定义 seek()函数

```
//跳转
void VideoPlayer::seek(int64_t pos) //精确到微秒
{
    if(!m_videoState.seek_req)
    {
        m_videoState.seek_pos = pos;
        m_videoState.seek_req = 1;
    }
}
```

第三个问题

第一篇文章我们就说过视频是每隔几秒才出现一个关键帧，而 av_seek_frame 只能跳转到关键帧,因此这里就会存在误差。举个例子：

假如 我们的视频 只有在 10 秒和 15 秒上才有关键帧，这时候我们希望跳转到 13 秒，调用

av_seek_frame 后视频只能跳到 10 秒或者 15 秒，因为解码器解码视频必须要有关键帧的信息才能解码，因此 av_seek_frame 这样设定是合理的，所以我们 seek 之后还需要执行一些操作来处理这个情况。那么实现就是将跳转转换到时间， 然后解码之后数据如果时间不到就舍弃掉， 不发送图片， 不播放声音.

在视频和音频的进程循环中添加

视频:

```
if (is->seek_flag_video)
{
    //发生了跳转 则跳过关键帧到目的时间的这几帧
   if (video_pts < is->seek_time)
   {
       av_free_packet(packet);
       continue;
   }else
   {
       is->seek_flag_video = 0;
   }
}
```

添加位置 计算 pts 后面， 位置如图

```
    else if (packet->dts != AV_NOPTS_VALUE)
    {
        video_pts = packet->dts;
    }
    else
    {
        video_pts = 0;
    }

    video_pts *= 1000000 *av_q2d(is->video_st->time_base);
    video_pts = synchronize_video(is, pFrame, video_pts);
    //跳转到关键帧,跳过一些帧
    if( is->seek_flag_video )
    {
        //发生跳转, 跳到关键帧的目的时间这几帧 中间的数据 丢弃
        if( video_pts < is->seek_time )
        {
            av_free_packet( packet );
            continue;
        }else // 到达目的时间
        {
            is->seek_flag_video  = 0 ;
        }
    }

    if (got_picture) {
        sws_scale(img_convert_ctx,
                (uint8_t const * const *) pFrame->data,
                pFrame->linesize, 0, pCodecCtx->height, pFrameRGB->data,
                pFrameRGB->linesize);
```

音频:

```
//跳转到关键帧,跳过一些帧
if( is->seek_flag_audio)
{
    if( is ->audio_clock < is->seek_time) //没有到目的时间
    {
        break;
```

```
    }
    else
    {
        is->seek_flag_audio = 0 ;
    }
```
}添加位置 计算音频时钟后面，位置如图
```
if( pkt.pts != AV_NOPTS_VALUE)
{
    is->audio_clock = pkt.pts *av_q2d( is->audio_st->time_base )*1000000 ; //取音频时
}else
{
    is->audio_clock = (*(uint64_t *) audioFrame->opaque)*av_q2d( is->audio_st->time_
}

//跳转到关键帧,跳过一些帧
if( is->seek_flag_audio)
{
    if( is ->audio_clock < is->seek_time) //没有到目的时间
    {
        break;
    }
    else
    {
        is->seek_flag_audio = 0 ;
    }
}
```

UI 界面添加跳转

自定义重写 QSlider 类型.
```
class VideoSlider : public QSlider
{
    Q_OBJECT

public:
    explicit VideoSlider(QWidget *parent = 0);
    ~VideoSlider();
signals:
    void SIG_valueChanged(int);
protected:
    void mousePressEvent(QMouseEvent *event);
};
```
只需要重写鼠标点击事件.
```
void VideoSlider::mousePressEvent(QMouseEvent *event)
{
    int value = QStyle::sliderValueFromPosition(
    minimum(), maximum(), event->pos().x(), width());
    setValue(value);
    emit SIG_valueChanged(value);
```

```
}
```
将 ui->slider_videoProcess 提升为 VideoSlider.
添加调整进度的信号槽连接
```
connect( ui->slider_videoProcess , SIGNAL(SIG_valueChanged(int)) , this,
SLOT(slot_videoSliderValueChanged(int)) );
//视频进度通过手动点击改变
void VideoShow::slot_videoSliderValueChanged(int value)
{
    if( QObject::sender() == ui->slider_videoProcess )
    {
        m_Player->seek((qint64)value*1000000);  //value 秒
    }
}
```